

Safe Memory Reclamation

Hazard Pointers and RCU: Two Answers to “When Is It Safe to Delete a Node” in a Lock-Free Data Structure

Robert (Bo) Hou · Robert’s Technical Notes · June 2026

Abstract. The first three reports (SPSC, Seqlock, MPMC) all concern *ordering and visibility* — data is written before it is read. But the moment a lock-free structure moves from “a circularly-reused array” to “a dynamically-allocated linked structure,” an entirely new, orthogonal problem detonates: **when is a piece of memory no longer in use by any thread, so that it can be safely deleted?** This is not an ordering problem; acquire/release is powerless against it. It is a *lifetime* problem. This report starts from a lock-free stack that crashes, letting use-after-free appear in person, then follows “why shared_ptr won’t do” to force out the two mainstream solutions: **hazard pointers** (each thread registers the pointer it is currently holding; the deleter scans before freeing) and **RCU** (readers merely mark entry/exit of a critical section; the writer waits one “grace period” and then reclaims in bulk). Both are given a mechanism derivation plus a correctness proof — the hazard-pointer proof is short and clean (a spatial criterion); the RCU proof is subtler (a temporal criterion) and its sufficiency rests on the conspiracy of two premises: the unreachability of the removed node and the non-retention outside a critical section. We close on a unifying view: both are hand-written, local garbage collectors that take the “when to reclaim” decision away from a runtime (as Java’s GC does) — which also explains why HFT prefers preallocated arrays: never deleting sidesteps the whole problem.

1. A Lock-Free Stack That Crashes: Use-After-Free Appears

Force the problem out first, rather than presenting the cure. Consider a lock-free stack with CAS-based head insertion/removal; both push and pop use the familiar CAS-retry pattern:

```

void push(T item) {
    Node* n = new Node{item, nullptr};
    n->next = head_.load(relaxed);
    while (!head_.compare_exchange_weak(n->next, n, release, relaxed)) { }
}

bool pop(T& out) {
    Node* old_head = head_.load(acquire);
    while (old_head != nullptr &&
           !head_.compare_exchange_weak(old_head, old_head->next, // (*)
                                         acquire, relaxed)) { }
    if (old_head == nullptr) return false;
    out = old_head->data;
    delete old_head; // (**)
    return true;
}

```

The ordering/visibility layer is correct: push publishes data with release, pop reads it with acquire on success — this happens-before pair guarantees “data is written before it is read.” But that layer cannot protect memory *lifetime*. Two threads both running pop suffice to break it:

The use-after-free interleaving (pure pop-pop; no push, no ABA). Stack: head → A → X → T1 executes `old_head = head_`, obtains A, then is suspended. T2 runs cleanly: CAS succeeds, unlinking A (head now points at X); it reads A’s data and delete A. A’s memory is freed. T1 wakes and continues its CAS: the second argument is `old_head->next = A->next` — **it must dereference A, but A has been deleted**. This happens before the CAS actually compares (the CAS argument must be evaluated first). Reading freed memory: undefined behavior.

The crux is line (*): `compare_exchange` is atomic, but its *argument* `old_head->next` is evaluated by ordinary code **before** entering the CAS, outside the atomic region. Atomicity protects the compare-and-swap from being interrupted; it cannot protect “the memory the pointer I am dereferencing still exists.” T1 holds A, T2 deleted A — two threads shared one `old_head`, one deleted it while the other was still using it. “Unlink and delete, heedless of who else holds it” is the root cause.

Two orthogonal problems, to be kept strictly apart. Ordering/visibility (acquire/release/happens-before) guarantees “the value read is correct”; **memory lifetime** (when it is safe to reclaim) guarantees “the address is still legal.” The former is the subject of the first three reports; the latter is this report’s. They are orthogonal: no amount of perfect acquire/release stops use-after-free. The array MPMC has no lifetime problem (slots are reused, never deleted); a linked structure (new/delete of nodes) makes it unavoidable.

2. Why shared_ptr Won’t Do: A Shared Count Is Contention

The first instinct is `shared_ptr`’s reference count: “don’t delete while someone still uses it.” The logic is right, but it cannot be used on a lock-free hot path, and the reason is worth

stating fully, because it foils exactly into the design motivation of both real solutions.

Problem 1: reading a shared `shared_ptr` is not itself thread-safe. Make `head` a `shared_ptr` and have many threads load/CAS it — its control block `refcount` is atomic, but the `shared_ptr` object itself (two pointers) is not read/written atomically; it tears. Safety needs `atomic<shared_ptr>` (C++20), which on many platforms degrades to a lock internally.

Problem 2 (fatal): a shared count is contention. The reference count is a single atomic counter *shared by all readers*. Every reader that loads a node must atomically increment/decrement it — so all readers hammer the same cache line, ping-ponging it across cores. The more reads, the worse the contention. In a read-heavy workload this is catastrophic. The core contradiction in one line: **a reference count makes “who is using it” a shared writable state, and shared writable = synchronization = contention.**

The shared breakthrough of both solutions. Hazard pointers and RCU take the same direction: **move “who is using it” from a shared writable count to per-thread private state.** A reader writes only its own copy (no other thread writes it) → the read side is contention-free. The cost shifts to the deleter, which bears “scan every thread’s private state / wait for every thread to pass through.” Since reads vastly outnumber deletes, this transfer pays off. The two differ in *what* the private state records: hazard pointers record “the specific pointer I am holding,” RCU records only “am I inside a critical section.”

3. Hazard Pointers: Register the Pointer, Scan Before Deleting

A hazard pointer makes “who is using it” a **per-thread private slot** that registers the pointer that thread is currently holding and does not want deleted.

- **Reader:** after obtaining a pointer (e.g. `old_head`), first **write it into its own hazard slot** (“I hold A”), then use it. A barrier after the store ensures the “registration” is visible to the deleter before the “use.”
- **Deleter:** before deleting node N, **scan all threads’ hazard slots**. If any slot == N, someone still holds N — **do not delete**; put N on a retired list and retry later. If no one registered it, delete safely.

Two engineering points: first, the number of pointers a thread protects at once is a **small constant** (a pop typically protects just one, `old_head`), so a few fixed atomic slots per thread suffice — no unbounded structure is needed; if a thread wants to register “a pile” of pointers, it is misusing hazard pointers. Second, “scan found a holder, cannot delete” must not drop the node — it goes onto a **per-thread private retired list**, reclaimed in bulk once a threshold is reached. This is deferred batch reclamation, amortizing cost.

Two tables, each with its job. Hazard table: one slot per thread, registering “the pointer I hold” — private write, global read. **Retired list:** one per thread, holding “nodes I want to delete but dare not yet” — private write, private read. Flow: reader `hazard[me] = ptr`; deleter unlinks a node → adds it to its own retired list → once full → scans all hazard tables → deletes those in retired not registered, keeps the rest. **All of this is transparent to the user:** the user calls only `protect(ptr)` and `retire(ptr)`; when the real delete happens is the library’s decision.

3.1 Correctness proof

The claim to prove: **when a node N is deleted, no thread still holds it**. The proof is short and clean — one invariant, one contradiction, two happens-before edges.

Theorem (hazard-pointer safety). Suppose the deleter, before deleting N, scanned all hazard tables and confirmed no slot == N (the invariant). Then when delete N executes, no thread holds a dereferenceable pointer to N.

Proof (contradiction). Suppose some thread T dereferences N after N is deleted. By the hazard-pointer contract, to dereference N, T must **first register N into its own hazard slot**, and this “registration” happens-before its “use” of N (guaranteed by the barrier after the store). Meanwhile the deleter’s “scan” happens-after its “unlink” of N. Hence, if T’s use occurs after the delete, and the registration precedes the use, then **at the moment the deleter scanned, T’s hazard slot already == N**. The deleter would have seen it and would **not** have deleted N — contradicting “N was deleted.” So no such T exists. ■

The linchpin is the two happens-before edges: “registration happens-before use” (reader’s barrier) and “unlink happens-before scan” (deleter’s ordering). This is a direct

application of the happens-before tooling honed in the earlier reports — hazard-pointer correctness is a spatial criterion (“is the pointer in the table”), which is why the proof is so clean.

4. RCU: Readers Only Mark Entry/Exit, the Writer Waits a Grace Period

The hazard-pointer read side still has a cost: every read writes its own slot plus a barrier. Where the read side must be *extremely* light (millions of reads per second, a write every few hours), that is still too dear. RCU's ambition is a read side of nearly **zero cost** — readers do not even register which pointer they hold.

4.1 The write side: copy-update-replace

The writer does not modify a node in place (in-place modification is a concurrent write; readers would see a half-update). Instead it **copies out a new version and atomically replaces a pointer**, making the old version unreachable from the structure. Take a **unidirectional tree (parent-to-child pointers only, no parent pointer)**: to change node, copy out node' (with the modification), then **atomically** swing its parent's child pointer from node to node'. Replacing a single pointer field is itself atomic, so the parent need not be copied; but if the parent also needs a multi-field update, copying propagates up the path, all the way to an **atomic swap of the root pointer** (path copying).

Why “no parent pointer” is a prerequisite, not an obstacle. If a node had a parent pointer, copying one node would require rewriting all its children's parent pointers, whose children are pointed at by others — bidirectional references make the copy cascade explode and never converge. A **unidirectional tree** (parent→child only) makes copying one node affect only “who points at it” (one inbound edge), not “what it points at” (children copied by reference), so the cascade travels up a single path and converges. And the writer, **traversing down from the root**, already holds the path, so it needs no parent pointer to find each level's parent to replace. This is exactly why persistent/functional structures are “parentless unidirectional trees.” RCU further requires the replacement to make the old node **fully unreachable** from the structure (single-inbound structures satisfy this naturally); multi-inbound (graphs/DAGs) need every inbound edge cleared first, or RCU does not directly apply.

Keep this distinct: the write side's **copy / path-copying / atomic root swap** is the *user's* data-structure update logic; “when it is safe to free the unlinked old node” is the **RCU library's** job, transparent to the user. The reclamation callback (e.g. `free_node`) only “frees this one old node” and has nothing to do with the copy path — the copy happened in the update phase (before unlink), reclamation happens after the grace period; the two are the opposite ends of a node's life.

4.2 The read side: only mark “am I in a critical section”

A reader enters a read section with `rcu_read_lock` and leaves with `rcu_read_unlock`. Crucially, this mark must be **per-thread private** (written by that thread, read by the writer), **not** a shared count (a shared count +1/-1 would fall back into the reference-count contention). It need not even be an explicit write: classic kernel RCU exploits “no context switch inside a read section,” so “a CPU underwent a context switch” by itself proves “its

preceding read section has ended” — zero explicit reader cost. Nesting is handled by a per-thread private depth counter (depth back to 0 = fully outside), no hashmap needed; different threads’ critical sections **need not agree** — different lengths, positions, code are all fine.

The counter-intuitive part: it says lock, but it is not a lock. `rcu_read_lock` does not block, does not exclude, grabs nothing — many readers may be inside critical sections at once; readers and readers, readers and the writer, are mutually non-exclusive. It is merely a very light mark recording “I am reading,” not a semaphore controlling “who may enter.” This is fundamentally different from a lock, where entry must queue.

4.3 The grace period and reclamation

At time T the writer unlinks node N (any **newly-started** traversal thereafter cannot reach N ; but readers already in flight before T may still hold N). The writer cannot delete immediately. It waits a **grace period: from T on, every thread passes through at least one moment “outside any critical section” (a quiescent point)**. Let T' be the moment the last thread does so; $T' \geq T$. The grace period ends at T' , and only then is N reclaimed.

Two reclamation APIs. **Synchronous** (`synchronize_rcu`): the writer blocks until the grace period ends, then frees itself (it may free several at once — they share one grace period). **Asynchronous** (`call_rcu(&node;, free_fn)`): non-blocking; hand “node + callback” to the library, which runs the callback in bulk after the grace period, in a dedicated reclamation context. **Multiple nodes share a single grace period**: whether one or a thousand were unlinked, so long as all were unlinked before T , one grace period reclaims them all — the root of RCU’s amortized batch reclamation.

4.4 Correctness proof

The claim: **reclaiming N after the grace period ends (T') leaves no reader holding N** . RCU’s proof is subtler than the hazard pointer’s — it is a *temporal* criterion, not a spatial one. First, the premises:

- **(A1)** A reader may obtain a node pointer only inside a critical section, from the *currently reachable* structure.
- **(A2)** A pointer obtained inside a critical section is no longer held after that section’s unlock.
- **(W1')** There is an unlink time T : traversals **newly started** after T cannot reach N ; readers already in flight before T are unaffected.

Theorem (RCU safety). Under (A1)(A2)(W1'), if reclamation runs at grace-period end T' (every thread passing at least one quiescent point after T), then no reader holds N at reclamation.

Proof. Fix any reader R ; let q be its first quiescent point after T (existing by the grace-period definition, with $q \leq T'$). Consider two cases for when R could hold N :

Case 1: the critical section in which R holds N began before T ($\text{lock} < T$). This section straddles T . Since q is a quiescent point (R is outside any critical section at q), and this section's $\text{lock} < T < q$, its unlock must be $\leq q \leq T'$ (else R would still be in this section at q , contradicting quiescence). By (A2), after unlock R no longer holds N.

Case 2: any critical section R enters after T ($\text{lock} \geq T$). By (W1'), traversals in these sections are newly started after T and **cannot reach N**; by (A1), R obtains pointers only from the reachable structure, so it **cannot obtain N at all** in these sections.

Combine. The two cases exhaust every interval in which R could hold N. In Case 1 R lets go before T' ; in Case 2 R never holds N. So after T' R does not hold N. R was arbitrary, so this holds for all readers. Therefore reclaiming N at T' is safe. ■

“Why one pass suffices” — the exact source of sufficiency. Sufficiency comes not from “passing through” itself, but from the **conspiracy** of two premises: **(W1')** underwrites Case 2 — the **unlink** means that even if a reader re-enters a critical section after passing, it cannot pick N back up; **(A2)** underwrites Case 1 — after unlock N is no longer held, so the straddling hold truly terminates. Neither alone suffices: without (W1'), a reader that passes and re-enters could grab N again, and it is never deletable; without (A2), the straddling hold does not end at unlock . **“Every thread passing one quiescent point after T ” exactly covers “every interval in which N could be held,” precisely because of these two premises.** This is the exact answer to where RCU “feels like something is still missing.”

5. A Comparison, and a Unifying View

Hazard pointers and RCU solve the same problem with orthogonal criteria.

- **Criterion type:** hazard pointers are a *spatial* criterion — scan before deleting to see “is this specific pointer present.” RCU is a *temporal* criterion — it never asks “which pointer do you hold,” only “after the unlink, has everyone passed through.”
- **Read-side cost:** hazard pointers write a slot plus a barrier per read; RCU is lighter, marking only entry/exit, and can even piggyback on natural events like context switches for zero explicit cost.
- **Write/reclaim cost:** hazard pointers scan all hazard tables before deleting; RCU's writer waits a grace period (possibly long), but many nodes share one grace period — batch reclamation is very cheap.
- **Granularity:** hazard pointers are fine (precise per-pointer protection); RCU is coarse (it only knows whether you are in a critical section). RCU trades per-pointer registration for the “unlink + pass” temporal guarantee — the very source of its lighter read side.

Unifying view: both are hand-written local garbage collectors. Their core question — “when is a piece of memory no longer used, so it can be reclaimed” — is exactly what garbage collection answers. A GC’d language (e.g. Java) **hands it to the runtime**: the runtime tracks reachability, reclaims the unreachable, and the user never manages pointer lifetimes (cost: stop-the-world pauses, unpredictable latency). Without a GC, C++ has no such backstop, so every lock-free structure **hand-writes a mini GC**: the hazard table / RCU critical-section mark = record “who still references it”; scan / wait a grace period = decide “is there still a reference”; retired list / call_rcu callback = “delete only once there is no reference.” And both are transparent to the user, who declares only retire / call_rcu while the runtime decides when — the signature of a GC.

This also explains why the **array-based bounded queue is so “simple.”** It reuses slots and never deletes, so it **sidesteps the entire safe-reclamation problem** — no new/delete means no “when to reclaim” hell, and it is immune to ABA for free (monotone integer indices, not reused pointers). This is the deep reason HFT prefers preallocation, slot reuse, and no new on the hot path: **the best solution to reclamation is to never create anything that needs reclaiming.** Avoid it where you can (arrays); where you cannot (linked structures/trees), choose between hazard pointers and RCU by “how light must the read side be” and “how long can the writer wait.”